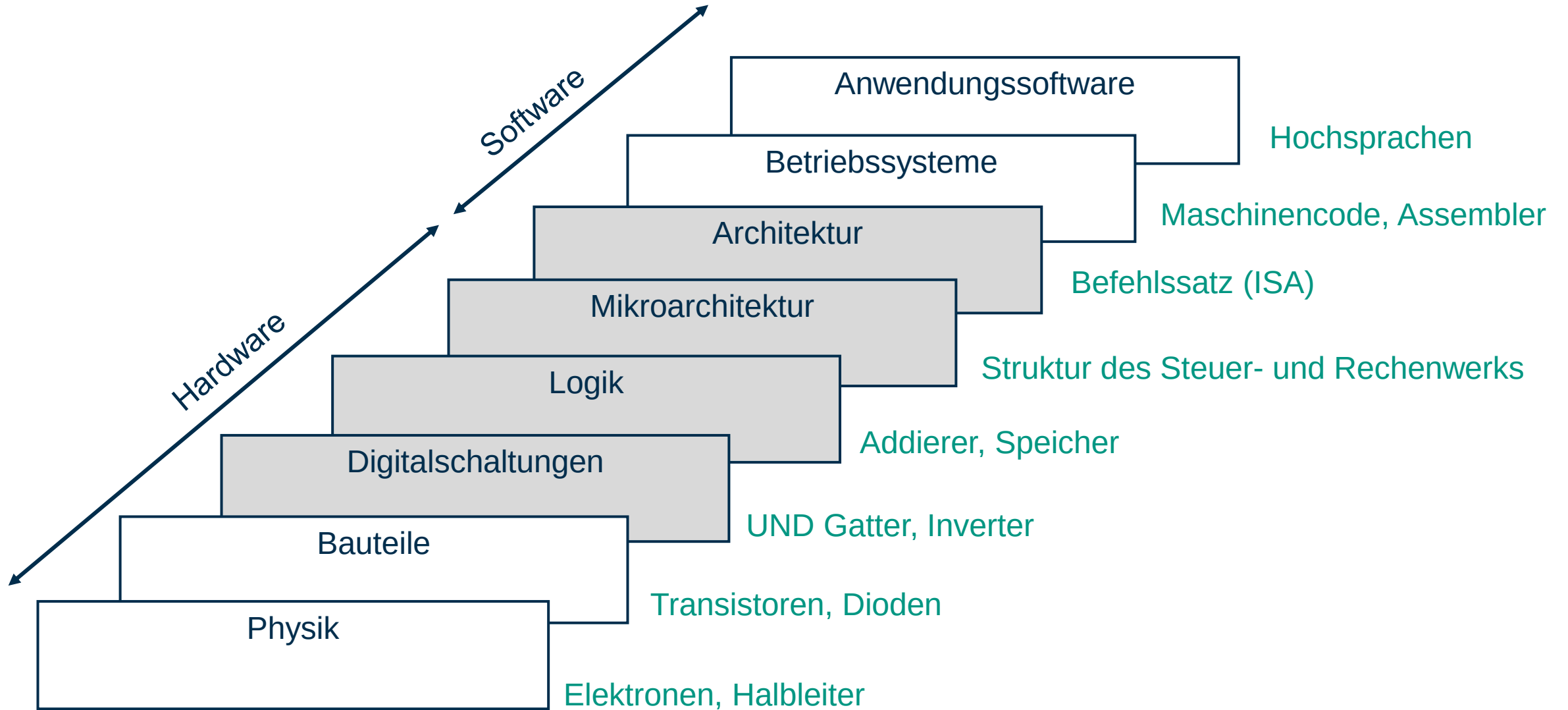


Informations verarbeitung

Codegenerierung und Abschätzung der Entwurfsqualität

Dr.-Ing. Dipl.-Inform. Tanja Harbaum
Institut für Technik der Informationsverarbeitung (ITIV)

Abstraktionsebenen



Übersicht – Vorlesung Nr. 02

- Compiler
 - Kurzeinführung
- General Purpose Processors
 - Instruction Set Architectures
 - Beispiel: RISC-V
 - Von C Code bis zum Maschinencode → Compiler
- Performanzmetriken
 - Übersicht Metriken: Latenz, Through-put, Auslastung, Energieeffizienz
 - Profiling und Tracing
 - Roofline Model

Compiler

- Ganz grundsätzlich: „Übersetzungswerkzeug von einer Programmiersprache auf Maschinencode“
- Einfache Compiler übersetzen Assembly-Code in einer Binärdarstellung für den Decoder der CPU

fe010113
00812e23
02010413
fea42623
fec42783
02f787b3
00078513
01c12403
02010113
00008067



addi	sp, sp, -32
sd	s0, 24(sp)
addi	s0, sp, 32
mv	a5, a0
sw	a5, -20(s0)
lw	a5, -20(s0)
mulw	a5, a5, a5
sext.w	a5, a5
mv	a0, a5
ld	s0, 24(sp)
addi	sp, sp, 32
jr	ra

Compiler

- Ganz grundsätzlich: „Übersetzungswerkzeug von einer Programmiersprache auf Maschinencode“
- Einfache Compiler übersetzen Assembly-Code in einer Binärdarstellung für den Decoder der CPU
- Fortgeschrittene Compiler übersetzen Hochsprachen (C, Java, Go, etc...) in einen Maschinencode für den CPU
 - Oft über Assembly-Code als „Zwischenschritt“

```
int square(int n){  
    return n * n;  
}
```

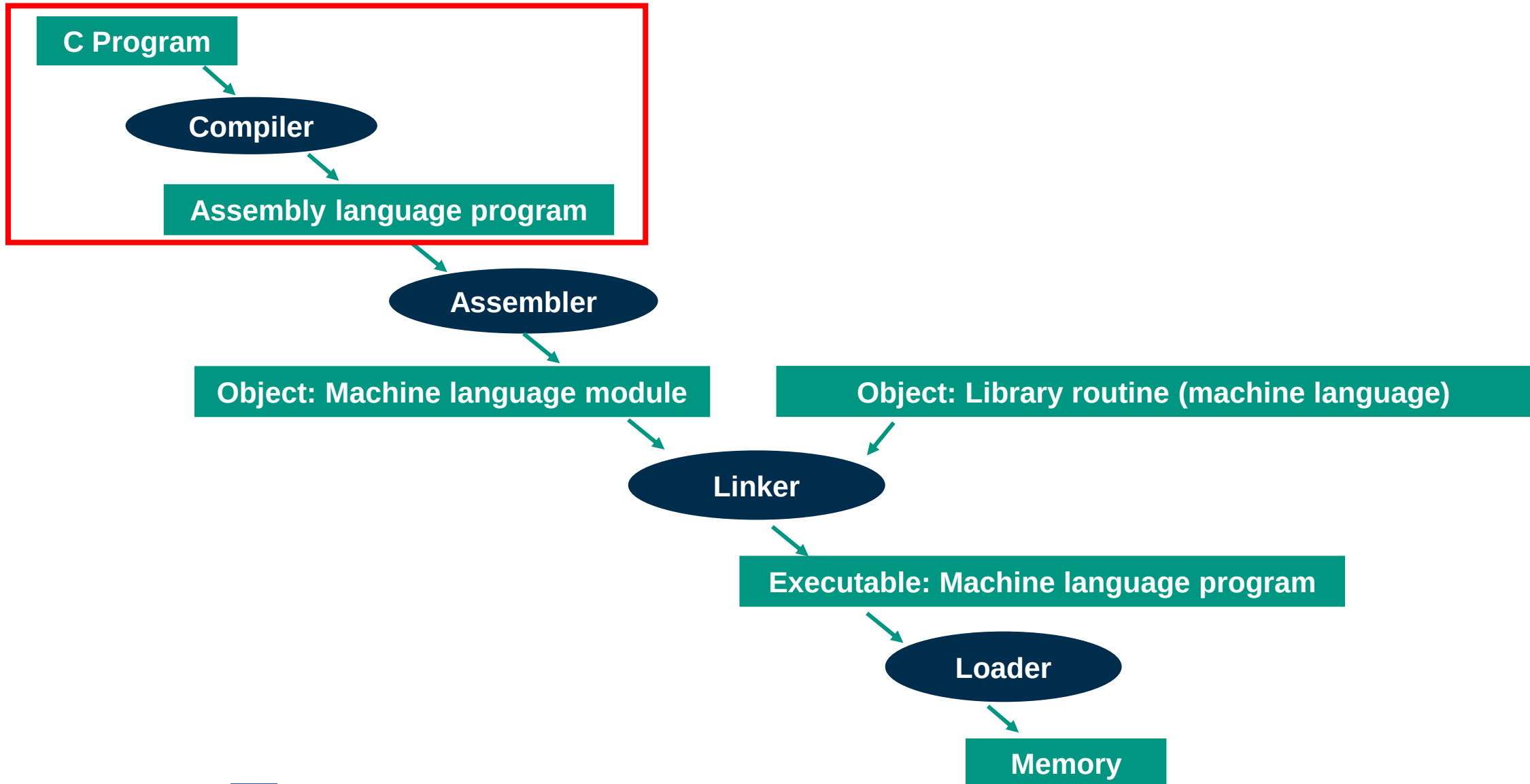


```
addi    sp, sp, -32  
sd      s0, 24(sp)  
addi    s0, sp, 32  
mv      a5, a0  
sw      a5, -20(s0)  
lw      a5, -20(s0)  
mulw    a5, a5, a5  
sext.w  a5, a5  
mv      a0, a5  
ld      s0, 24(sp)  
addi    sp, sp, 32  
jr      ra
```

```
fe010113  
00812e23  
02010413  
fea42623  
fec42783  
02f787b3  
00078513  
01c12403  
02010113  
00008067
```



Compiler Workflow



Compiler Workflow

```
void main() {  
    int x5 = 5;  
    int x6 = 5;  
    int x7;  
  
    if (x5 == x6) {  
        x7 = 1;  
    } else {  
        x7 = 99;  
    }  
}
```

C Program

Compiler



GNU Compiler
Collection

Assembly language program

```
_start:  
    addi x5, x0, 5  
    addi x6, x0, 5  
    beq x5, x6, match  
    addi x7, x0, 99
```

```
match:  
    addi x7, x0, 1
```

Compiler Workflow

Assembly language program

Assembler

Object: Machine language module

```
_start:  
    addi x5, x0, 5  
    addi x6, x0, 5  
    beq x5, x6, match  
    addi x7, x0, 99
```

```
match:  
    addi x7, x0, 1
```



Teil von GNU Binutils

```
0x00500293  
0x00500313  
0x00628663  
0x06300393  
0x00100393
```


Compiler Workflow

Object: Machine language module

0x00500293
0x00500313
0x00628663
0x06300393
0x00100393

Object: Library routine
(machine language)

Linker



Gnu Binutils enthält
auch Linker

Executable: Machine
language program

```
00000000 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010 02 00 F3 00 01 00 00 00 B0 00 01 00 00 00 00 00
00000020 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030 04 00 00 00 40 00 38 00 02 00 00 00 00 00 00
00000040 01 00 00 00 05 00 00 00 00 00 00 00 00 00 00
00000050 00 00 01 00 00 00 00 00 00 00 01 00 00 00 00
00000060 CC 00 00 00 00 00 00 00 CC 00 00 00 00 00 00
00000070 00 10 00 00 00 00 00 00 01 00 00 00 06 00 00
00000080 CC 00 00 00 00 00 00 00 CC 10 01 00 00 00 00
00000090 CC 10 01 00 00 00 00 00 00 00 00 00 00 00 00
000000A0 00 00 00 00 00 00 00 00 00 10 00 00 00 00 00
000000B0 93 02 50 00 13 03 50 00 63 86 62 00 93 03 06
000000C0 6F 00 80 00 93 03 10 00 6F 00 00 00
```

Compiler Workflow

Executable: Machine
language program

Loader

Memory

```
00000000 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00
00000010 02 00 F3 00 01 00 00 00 B0 00 01 00 00 00 00
00000020 40 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030 04 00 00 00 40 00 38 00 02 00 00 00 00 00
00000040 01 00 00 00 05 00 00 00 00 00 00 00 00 00
00000050 00 00 01 00 00 00 00 00 00 00 01 00 00 00
00000060 CC 00 00 00 00 00 00 00 CC 00 00 00 00 00
00000070 00 10 00 00 00 00 00 00 01 00 00 00 06 00
00000080 CC 00 00 00 00 00 00 00 CC 10 01 00 00 00
00000090 CC 10 01 00 00 00 00 00 00 00 00 00 00 00
000000A0 00 00 00 00 00 00 00 00 10 00 00 00 00
000000B0 93 02 50 00 13 03 50 00 63 86 62 00 93 03
000000C0 6F 00 80 00 93 03 10 00 6F 00 00 00
```

0x00000000

127

0x00000001

69

0x00000002

76

0x00000003

70

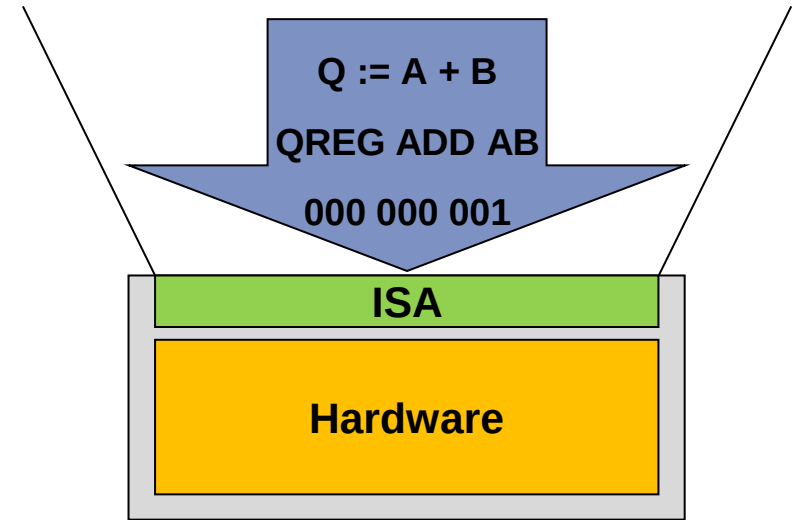
0x00000004

2

Compiler Zusammengefasst

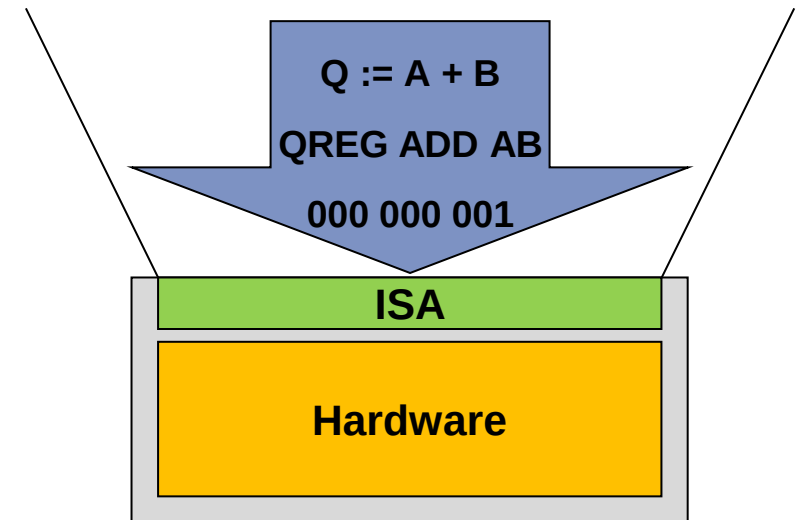
- „Übersetzungswerkzeug von einer Programmiersprache auf Maschinencode“
- Woher weiß der Compiler wie die Assembly Instruktionen codiert werden?

→ Instruction Set Architecture (Befehlssatzarchitektur)



Instruction Set Architectures (ISA)

- Eine ISA beschreibt die nach außen sichtbare Funktion / Architektur eines Prozessors
 - Was ist die Kodierung für z.B. einer Additions-Operation?
→ Befehlssatz
 - Welche Register gibt es?
 - Wie groß sind die Operanden in Bit?
 - Wie werden Operanden geladen / gespeichert?

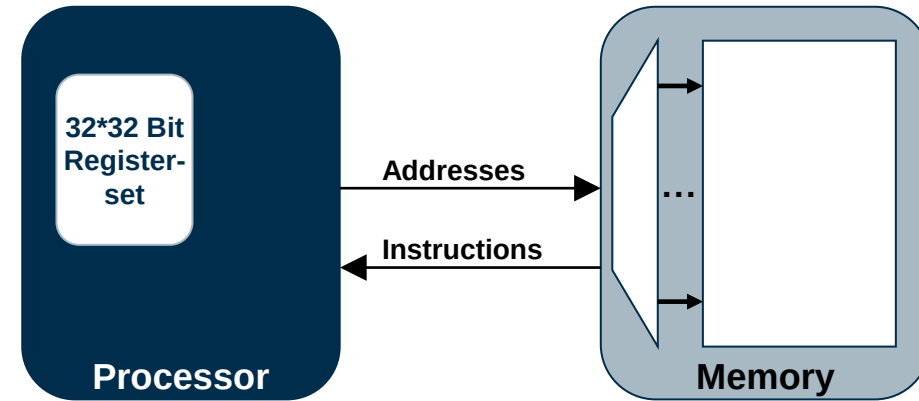


Grundtypen von verschiedenen ISAs

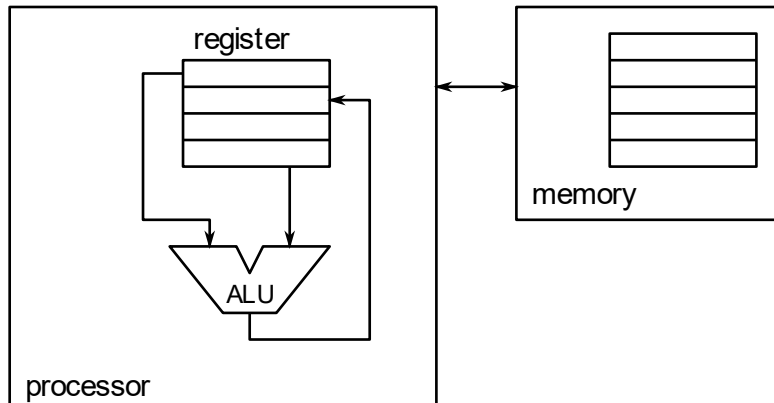
- CISC „Complex Instruction Set Computing“
 - Beispiele: Intel x86, AMD64
 - https://en.wikibooks.org/wiki/X86_Assembly
- RISC „Reduced Instruction Set Computing“
 - Beispiele: ARM Cortex Familie, RISC-V
 - <https://developer.arm.com/documentation/dui0646/c/The-Cortex-M7-Instruction-Set/Instruction-set-summary>
- VLIW „Very Long Instruction Word“
 - Beispiele: TI C6000 DSP Familie
 - <https://www.ti.com/lit/ug/spru733a/spru733a.pdf>
- EPIC „Explicitly Parallel Instruction Computing“
 - Beispiele: Intel Itanium IA-64

Reduced/Complex Instruction Set Computer

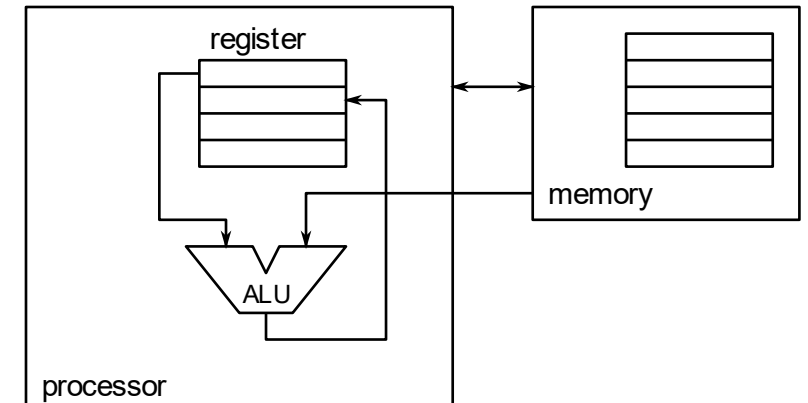
■ Struktur



■ RISC



■ CISC



Reduced/Complex Instruction Set Computer

Reduced instruction set computer (RISC)

- Nur ein Taktzyklus pro reduzierter Instruktion
- Load-Store-Architektur
 - Load-Store sind unabhängige Instruktionen
- Große Codegrößen
- Benötigt mehr Transistoren für Speicherregister
- Schwerpunkt auf Software

- Beispiel: $c := a + b$
 - LOAD R1, a
 - LOAD R2, b
 - ADD R3, R2, R1
 - STORE c, R3

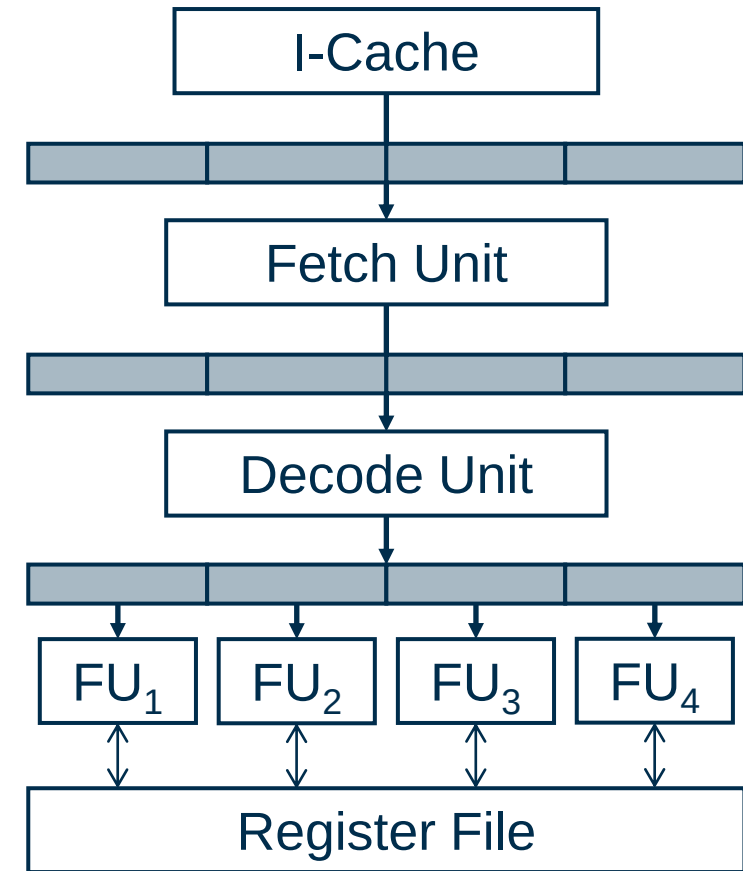
Complex instruction set computer (CISC)

- Mehrere Taktzyklen pro komplexer Instruktion
- Memory-to-Memory-Architektur
 - Load/Store sind eingebaute Instruktionen
- Kleine Codegrößen
- Benötigt mehr Transistoren zum Speichern komplexer Instruktionen
- Schwerpunkt auf Hardware

- Beispiel: $c := a + b$
 - MOVE R1, a
 - ADD R1, b
 - MOVE c, R1

Very Long Instruction Word (VLIW)

- Befehlspaket mit fester Anzahl von Befehlen
 - Parallelität auf Befehlsebene
 - Ein Maschinenbefehlswort fester Länge
 - Typischerweise zwischen 64 -1024 Bit
 - voneinander unabhängige Befehle
 - Jeder eine Befehl ist einer eigenständigen Funktionseinheit zugeordnet
 - Alle Funktionseinheiten nutzen eine gemeinsame Registerdatei



Instruction Set Architectures (ISA)

- Einteilung verschiedener ISAs
 - Grundtypen RISC, CISC, VLIW, ...

Instruction Set Architectures (ISA)

- Einteilung verschiedener ISAs
 - Grundtypen RISC, CISC, VLIW, ...
 - Bitbreite
 - Wie groß sind die Operanden und Register?
 - Desktop-Anwendungen: Typisch 64-bit
 - Low-Cost / Low-Power Anwendungen wie Mikrocontroller: 8-bit oder 16-bit

Instruction Set Architectures (ISA)

- Einteilung verschiedener ISAs
 - RISC (Reduced Instruction Set Computer) oder CISC (Complex Instruction Set Computer)
 - Bitbreite
 - Unterstützte Funktionalität
 - Welche Operationen werden unterstützt?
 - Gibt es die Möglichkeit Fließkommazahlen zu berechnen?
 - Gibt es doch einige Spezialoperationen?

ISA – Typische Operationen

- Arithmetik / Logik
 - Addition, Subtraktion, Multiplikation, ggf. Division, Inkrement
 - Logische Operationen: AND, OR, XOR, ...
 - Addition mit Carry, Subtraktion mit Carry
 - Shift-Operationen: z.B. alle Bits in einem Register nach links / rechts schieben

ISA – Typische Operationen

- Arithmetik / Logik
- Kontrollfluss
 - Compare (Vergleicht zwei Operanden und setzt die Zero-Flag auf 0)
 - Jump (Zum „Springen“ zu einer Unterfunktion)
 - Kombination von beiden ermöglicht Kontrollfluss wie „if“-Abfragen
 - In RISC-V sind die Sprünge häufig bereits Kombinationen (Beispiel: branch if equal)

ISA – Typische Operationen

- Arithmetik / Logik
- Kontrollfluss
- Datenbewegung
 - Load / Store: Lesen / Speichern vom / zum Arbeitsspeicher
 - Laden von festen Werten in Register
 - Laden / Schreiben auf den Stack: Push / Pop

ISA – Typische Operationen

- Arithmetik / Logik
- Kontrollfluss
- Datenbewegung
- Weitere Instruktionen
 - Clear / Set Flag: z.B. Carry-Flag löschen
 - NOP: No Operation
 - WFI: Stoppt den Prozessor bis zu einem nächsten Interrupt

- RISC-V ist eine Load-Store-Architektur
 - Load- und Store-Befehle greifen auf den Speicher zu
 - Ein Load-Befehl liest einen Wert aus dem Speicher in ein Register
 - Ein Store-Befehl schreibt einen Wert aus einem Register in den Speicher
 - Andere Befehle arbeiten mit CPU-Registern
- RV32 ist eine 32-Bit-Architektur
 - Alle arithmetischen Operationen werden auf 32-Bit-Wörtern durchgeführt
 - RISC-V besitzt 32 Register
- RISC-V verwendet Byte-Adressierung
 - Standard bei CPUs

RISC-V[®] ISA - Beispiele von typischen Operationen

■ Arithmetische Operationen

- Addition von zwei Registerinhalten: `add rd, rs1, rs2`
- Subtraktion von zwei Registerinhalten: `sub rd, rs1, rs2`
- Addition konstanten Wert (Immediate) direkt zu einem Registerinhalt: `addi rd, rs1, imm`

■ Kontrollfluss

- Programmfluss ändern (PC+= imm), wenn zwei Werte gleich sind: `beq rs1, rs2, imm`

■ Datenbewegung

- Konstanten Wert (Immediate) in ein Register laden: `li rd, imm`
- 32-Bit-Wort aus RAM in ein Register laden: `lw rd, imm(rs1)`

■ RISC-V Assembler Cheat Sheet:

- <https://projectf.io/posts/riscv-cheat-sheet/>

Die Rolle von Konventionen

- Konventionen erweitern ISA um zusätzliche Regeln
 - Für RISC-V dokumentiert unter <https://github.com/riscv-non-isa>
 - Konventionen fügen keine neuen Befehle hinzu und verändern die RISC-V ISA nicht
 - Sie helfen ein Ökosystem um die ISA Spezifikation aufzubauen
 - ABI-Dokumentation (Application Binary Interface)
 - Reservierung von bestimmten Prozessorregistern für bestimmte Zwecke, die Richtung des Stacks oder das Format von Gleitkommazahlen
 - Regelt, wie Programme mit dem Betriebssystem und untereinander interagieren
 - Architekturtests und vieles mehr

Die Rolle von Konventionen

- Konventionen erweitern ISA um zusätzliche Regeln
 - Für RISC V dokumentiert unter <https://github.com/riscv-non-isa>

- Beispiel: Return Address

Name	ABI Mnemonic	Meaning
x1	ra	Return address

- In Assembly werden viele Sprung Instruktionen benötigt
- Die Return Address speichert die Stelle an die wir später zurückkehren wollen
- ➡ Die Rückkehr Adresse wird per Konvention in Register x1 (ra) gespeichert

RISC-V® - RV32 ABI Register

- Beispiel für Anforderungen:
 - Menge an Registern (32)
- Beispiel für Konventionen:
 - Zweck/Benennung der Register

Name	ABI Mnemonic	Meaning
x0	zero	Zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7	t0-t2	Temporary registers
x8-x9	s0-s1	Callee-saved registers
x10-x17	a0-a7	Argument registers
x18-x27	s2-s11	Callee-saved registers
x28-x31	t3-t6	Temporary registers

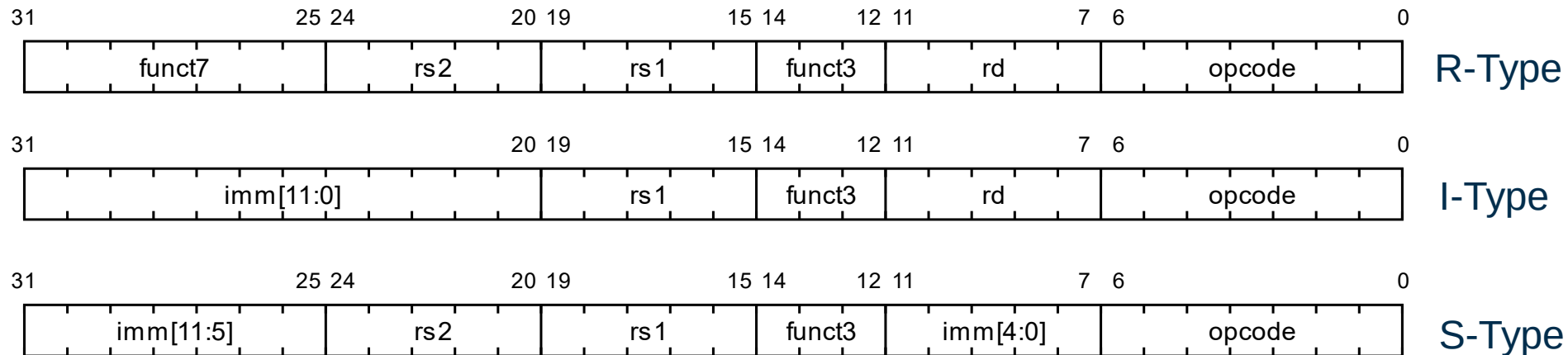
RISC-V Register Konvention

- RISC-V verfügt über einen modularen Aufbau
 - Verschiedenen Basiskomponenten
 - Optionale Erweiterungen
- Jede RISC-V Architektur muss einen Integer Befehlssatz implementieren
 - RV32I / RV64I für 32- / 64-bit Register
 - RV32E / RV64E reduzierte Varianten mit weniger Integer Registern (16 statt 32)
- Standard Integer Befehlssätze enthalten nur Addition/Subtraktion, Speicherbefehle und Kontrollbefehle

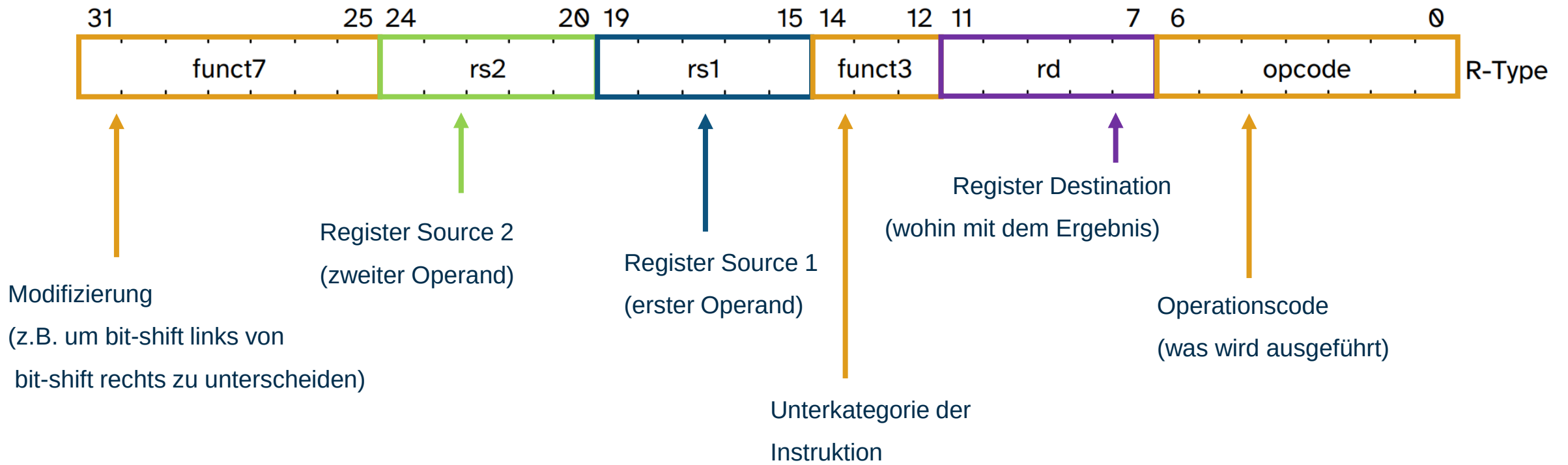
- Welche Erweiterungen implementiert werden ist frei wählbar

Name	Anzahl Instruktionen	Art	Beschreibung
RV32I	40	Pflicht	Ganzzahl-Arithmetik, Lade-/Speicherooperationen und Kontrollfluss
M	8 / 13	Optional	Ganzzahlige Multiplikation & Division
A	11 / 22	Optional	Atomare Instruktionen
C	40	Optional	Komprimierte Instruktionen, 16-bit
F	26 / 30	Optional	Floating-point Arithmetik

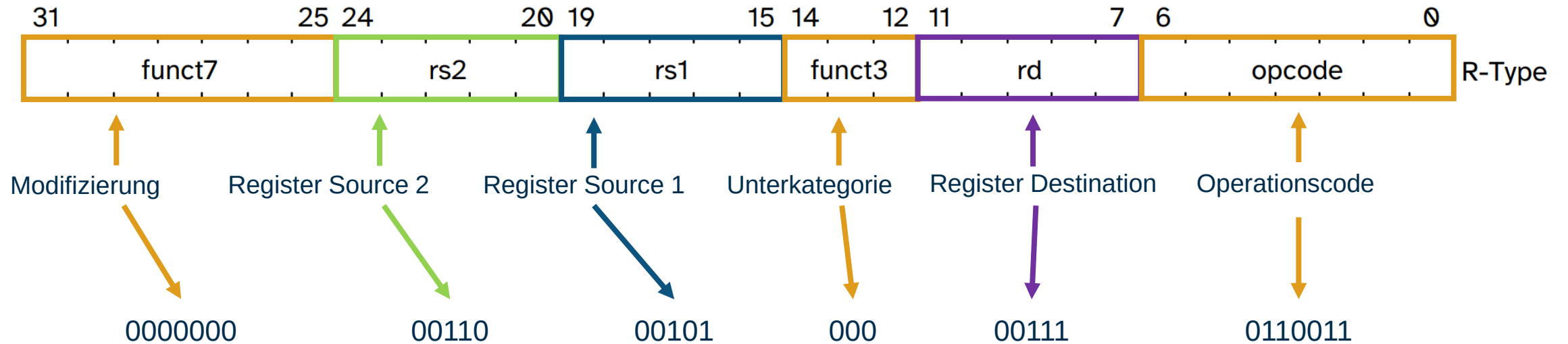
- Instruktionen können verschiedenen Strukturen folgen
 - R-Type Register-Register Operationen
 - I-Type Immediate Operationen
 - S-Type Store Operationen
 - Außerdem: U-Type, B-Type, J-Type



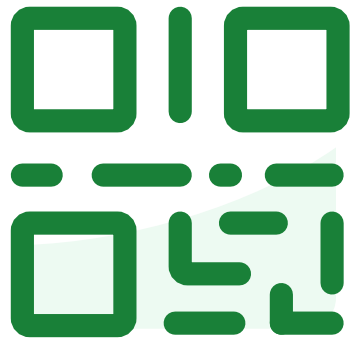
RISC-V® R-Type Instruktionen



RISC-V R-Type Instruktionen



Beispiel: **add** **x7**, **x5**, **x6**



**Join at slido.com
#1732274**



**Welche der Instruktionen
speichert ihr Ergebnis in x3?**

RISC-V[®] ISA - Beispiele von typischen Operationen

■ Arithmetische Operationen

- Addition von zwei Registerinhalten: `add rd, rs1, rs2`
- Subtraktion von zwei Registerinhalten: `sub rd, rs1, rs2`
- Addition konstanten Wert (Immediate) direkt zu einem Registerinhalt: `addi rd, rs1, imm`

■ Kontrollfluss

- Programmfluss ändern (PC+= imm), wenn zwei Werte gleich sind: `beq rs1, rs2, imm`

■ Datenbewegung

- Konstanten Wert (Immediate) in ein Register laden: `li rd, imm`
- 32-Bit-Wort aus RAM in ein Register laden: `lw rd, imm(rs1)`

■ RISC-V Assembler Cheat Sheet:

- <https://projectf.io/posts/riscv-cheat-sheet/>



**Welche der Optionen berechnet
 $5+3$?**

- Wie berechnen wir $5+3$?

`addi t0, x0, 5` ← Addiere 0 und 5,
`addi t1, x0, 3` speichere das Ergebnis in t0
`add t2, t0, t1`

addi für add immediate

RISC-V Register Konvention

Name	ABI Mnemonic	Meaning
x0	zero	Zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7	t0-t2	Temporary registers
x8-x9	s0-s1	Callee-saved registers
x10-x17	a0-a7	Argument registers
x18-x27	s2-s11	Callee-saved registers
x28-x31	t3-t6	Temporary registers



Welchen Wert hält t_2 am Ende?

- Wie berechnen wir $7-4+2$?

```
addi  t0, x0, 7
addi  t1, x0, 4
sub   t2, t0, t1
addi  t3, t2, 2
```

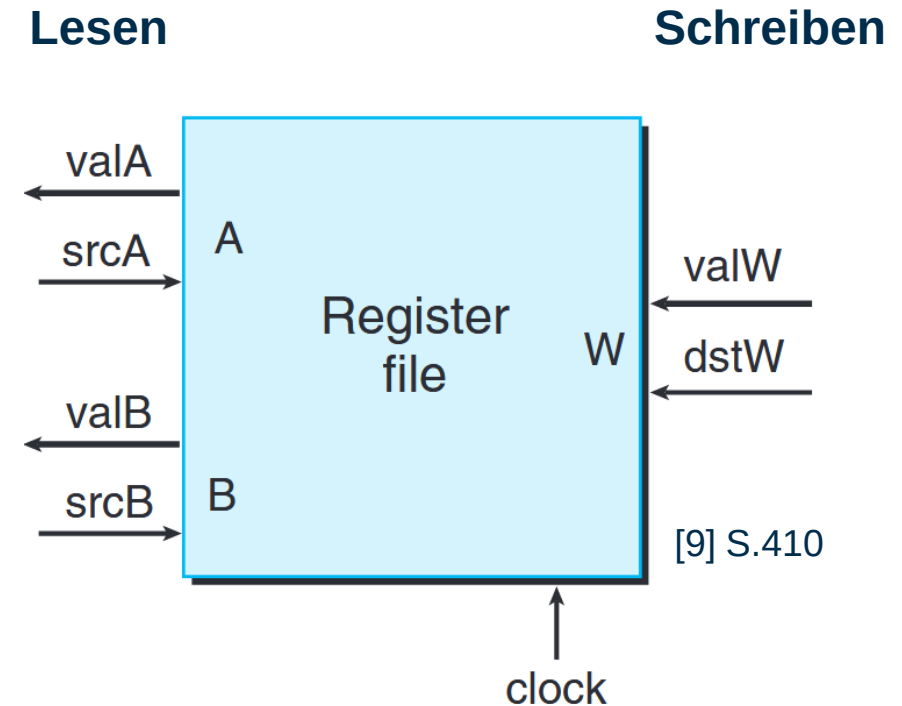

Warum gibt es so wenige Register?

- Die Registeradressen sind ein fester Teil aller Instruktionen
 - 5 Bit pro Registeradressierung
 - Für zusätzliche Register werden Instruktionen länger (weniger Code-Dichte)
 - Verschiedene Opcode Varianten möglich, Decoder-Komplexität wird jedoch erhöht
- Jedes zusätzliche Register erhöht die Hardwarekosten
 - Ansteuerung wird komplexer
- Moderne Compiler können mit 32 Registern bereits den Großteil der benötigten temporären Werte speichern
 - Zusätzliche Register würden nur marginale Verbesserungen bringen
- Erweiterbarkeit bei Bedarf
- Bewusstes Design-Trade-off
 - Einfache Instruktionscodierung, geringe Hardware-Kosten und ausreichend Compiler-Leistung

Typischer Registeraufbau

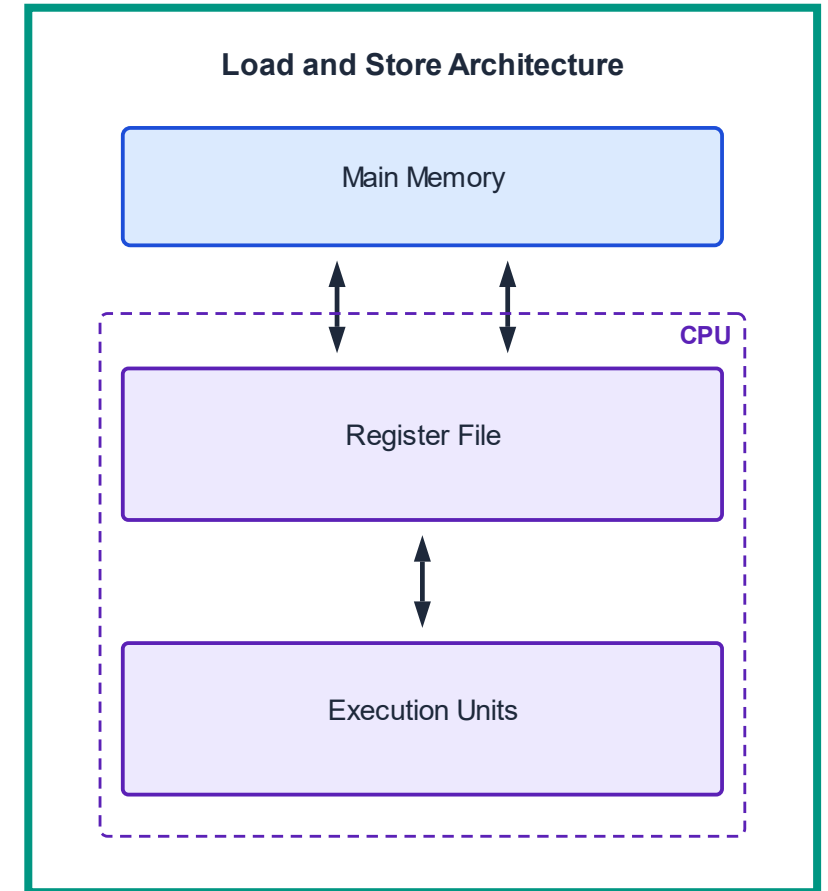
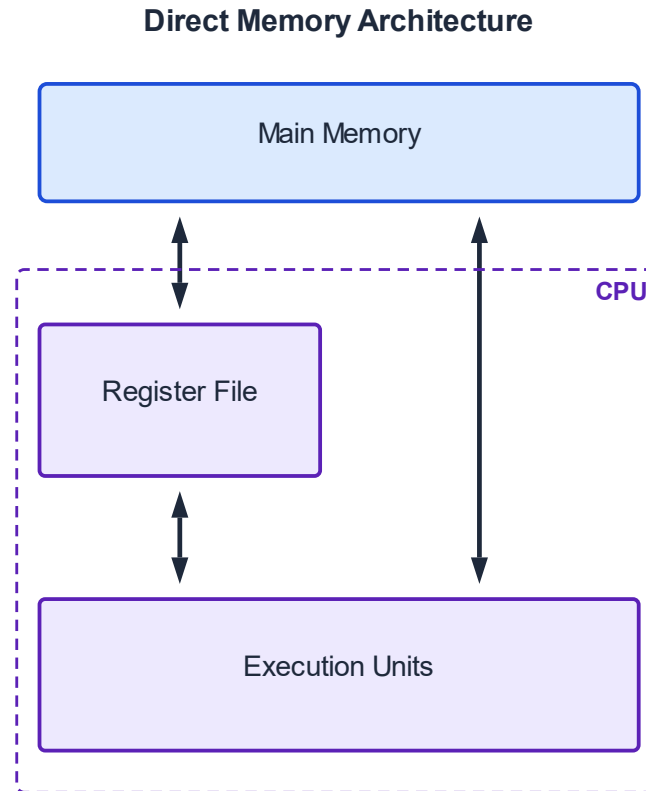
- Die Menge an Registern wird als „Register file“ gruppiert
- Es gibt typischerweise mehr Ports zum Lesen als zum Schreiben (In diesem Beispiel 2:1)
- Somit können bspw. Additionen in einem Taktzyklus ausgeführt werden

add **t2**, **t0**, **t1**



RISC-V[®] Memory Access

- RISC-V ist eine load-store Architektur
- Werte aus dem Speicher werden zuerst in Register geladen



[11]

- Je nach Wert gibt es andere Instruktionen zum Laden (z.B. lb load byte)
- Für jede Größe gibt es ein passendes Gegenstück mit s (z.B. sb store byte)*

Instruktion	Kürzel	Beschreibung
Load Word	lw	Lädt 32-Bit Wert aus dem Speicher
Load Halfword	lh	Lädt 16-Bit Wert aus dem Speicher
Load Byte	lb	Lädt 8-Bit Wert aus dem Speicher
Load Halfword Unsigned	lhu	Lädt 16-Bit Unsigned Wert aus dem Speicher (vorzeichenlos)
Load Byte Unsigned	lbu	Lädt 8-Bit Unsigned Wert aus dem Speicher (vorzeichenlos)

*es gibt keine separaten Store Instruktionen für signed/unsigned Werte

RISC-V® Memory Access (RV32)

- Beispiel: Lade **drittes** Array Element aus dem Speicher
- Angenommen **t0** hält die Speicheradresse des ersten Elements

lw **t1, 8(t0)**

load word 2x 4 Byte Offset

t0

0x00000000

0x00000004

0x00000008

0x0000000C

0x00000010

0x00000014

0x00000018

0x0000001C

0x00000020

0x00000024

0x00000028

325

2

3

5

7

11

13

17

19

4550

12



Im Speicher befindet sich ein Array mit den Zahlen 0 bis 9 als 32-bit Integers. t0 hält die Adresse von dem ersten Element. Welchen Wert hält t1 nach der Instruktion `lw t1, 32(t0)`

RISC-V[®] Memory Access (RV32)

- Beispiel: Wert an Stelle **0x00000010** laden, **4** addieren und das Ergebnis an der selben Adresse speichern

```
lw    t1, 12(t0)
addi  t1, t1, 4
sw    t1, 12(t0)
```

t0

0x00000000

0x00000004

0x00000008

0x0000000C

0x00000010

0x00000014

0x00000018

0x0000001C

0x00000020

0x00000024

0x00000028

325

2

3

5

7

11

13

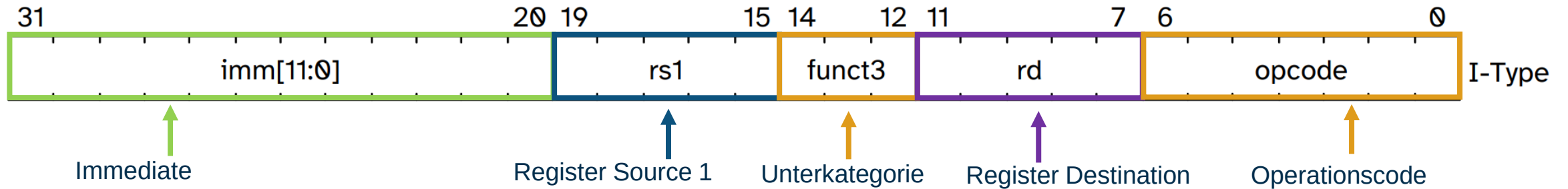
17

19

4550

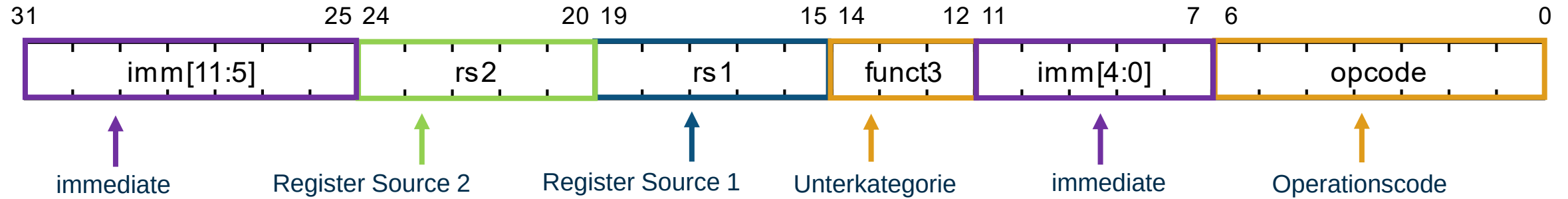
12

I-Type Instruktionen



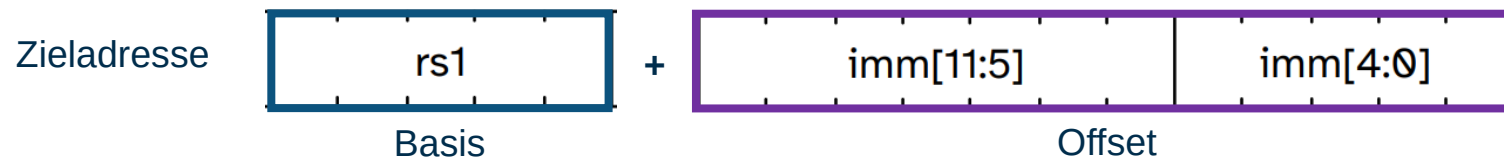
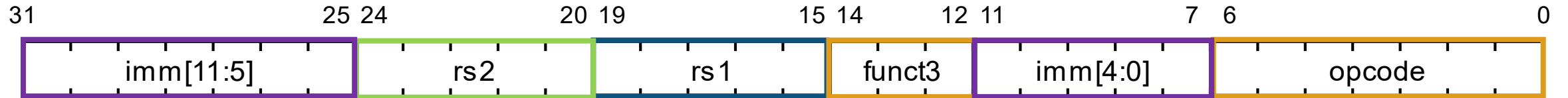
- Immediate Operationen
- Wir kennen bereits ein Beispiel: **addi**
- Andere Beispiele: **slti** (Set less than immediate), **andi**, **ori** (bitweise Logik)
- Load Instruktionen wie **lw** folgen demselben Format

S-Type Instruktionen

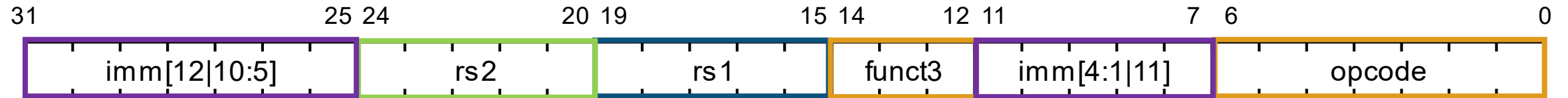


- Store Operationen
- Wir kennen bereits ein Beispiel: **sw**
- andere Beispiele: **sh** (store half-word), **sd** (store doubleword), **sb** (store byte)
- Wie kann man die Instruktion deuten?

S-Type Instruktionen

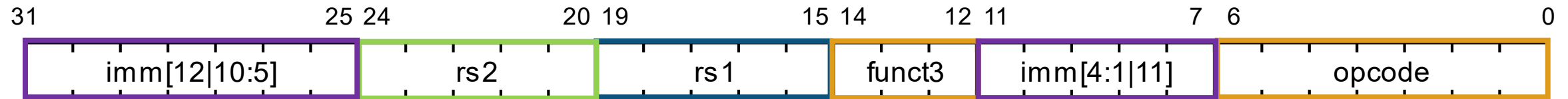


B-Type Instruktionen



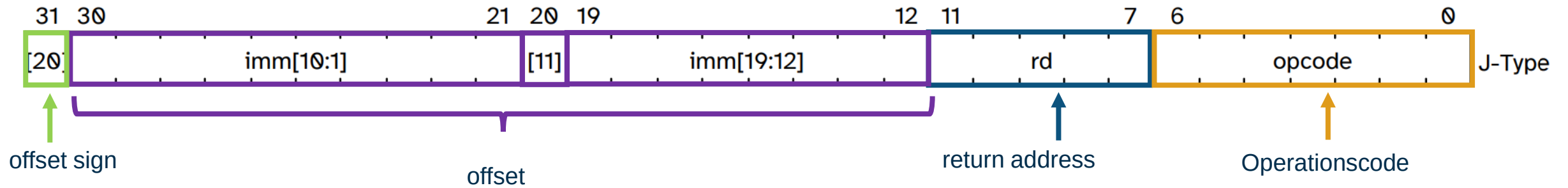
- Branch (Abzweigung) Instruktionen
- Nützlich für logische Operationen wie if else statements
- Auch gut für loops

B-Type Beispiel: beq



- **beq** (branch if equal) macht einen Sprung wenn die Werte in den Registern gleich sind
- Beispiel: **beq** **x5**, **x6**, destination
- Springt an die Adresse „destination“ wenn die Register x5 und x6 denselben Wert enthalten

J-Type Instruktionen



- Es gibt nur eine J-Type Instruktion **jal** (Jump and Link)
- Ziel ist der Sprung von einer Stelle des Program Counters zu einer anderen (1 MiB Reichweite)

Was macht überhaupt jal?

- jal steht für Jump And Link `jal rd, imm`
- der Program Counter wird um einen Offset verringert/erhöht ($PC \pm imm$)
- die Return Address ($rd = PC+4$) wird das angegebene Register geschrieben

Übersicht – Vorlesung Nr. 02

- Compiler
 - Kurzeinführung
- General Purpose Processors
 - Instruction Set Architectures
 - Beispiel: RISC-V
 - Von C Code bis zum Maschinencode → Compiler
- Performanzmetriken
 - Übersicht Metriken: Latenz, Throughput, Auslastung, Energieeffizienz
 - Profiling und Tracing
 - Roofline Model

Latenz & Durchsatz Motivation

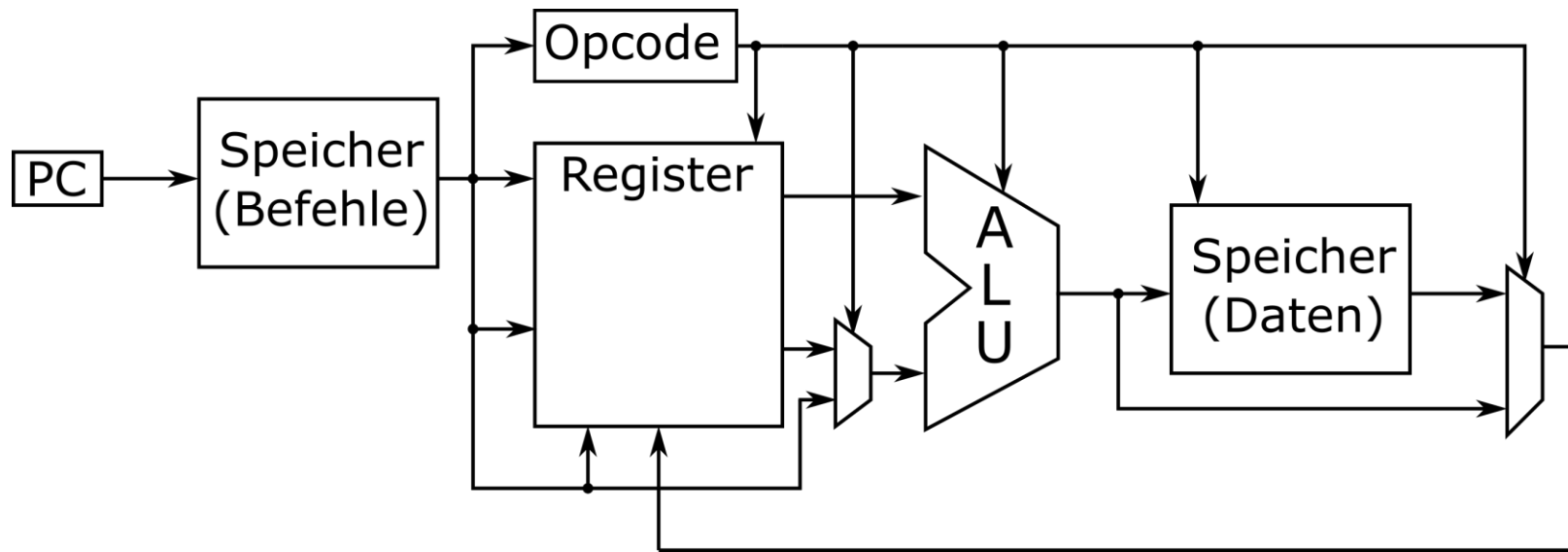
- Latenz: Wie viel Sekunden braucht ein Auto um den Tunnel zu durchqueren
- Durchsatz wie viele Autos durchqueren den Tunnel pro Sekunde



Generiert mit Gemini

Latenz & Durchsatz Allgemein

- Latenz: Wie viel Zeit braucht die Ausführung einer Operation
- Durchsatz: Wie viele Operationen können pro Zeiteinheit ausgeführt werden





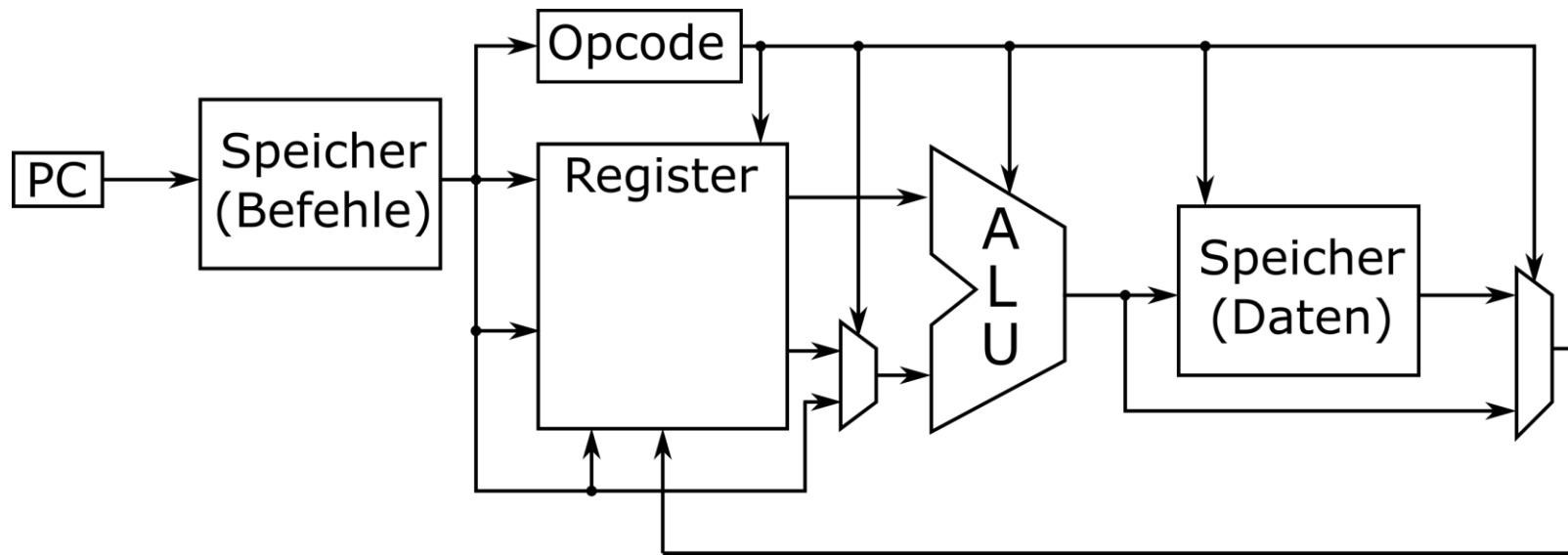
Ein neuronales Netz klassifiziert Bilder mit einer Latenz von 0.5s. Wie hoch ist der Durchsatz?



**Ein neuronales Netz klassifiziert 5
Bilder/s. Wie hoch ist die Latenz?**

Latenz & Durchsatz Allgemein

- Latenz: Wie viel Zeit braucht die Ausführung einer Operation
- Durchsatz: wie viele Operationen können pro Zeiteinheit ausgeführt werden



- Durchsatz und Latenz stehen nicht im direkten Zusammenhang
 - In Hardwarelösungen stehen sie aber in der Regel im Tradeoff zueinander

Verlustleistung und Energie

- Leistung (power dissipation)
 - $P [W]$
 - Wichtig für Dimensionierung von: Packaging, Power Supply, Cooling
- Energie (power-delay product)
 - $E = P_{avg} * T_{exec} [Ws]$
 - Wichtig für mobile Geräte (Batterie/Akku - Lebenszeit).
 - Sinnvolles Maß bei Systemen, die mit einer festen Rate betrieben werden.
 - Energie / Instruktion
 - Leistung auf einen Taktzyklus bezogen: $[\mu W/MHz]$
 - bei Prozessoren oft:
 - $[\mu W/MIPS]$ or $[MIPS/\mu W]$
 - $[\mu W/SPEC]$ or $[SPEC/\mu W]$

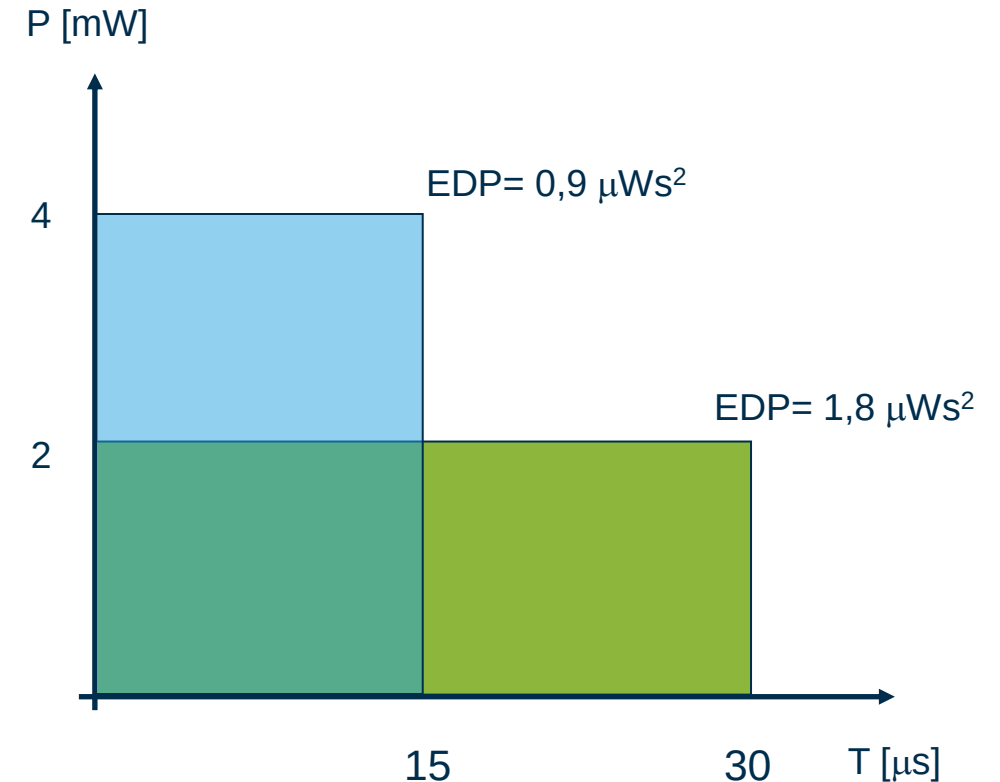
Energieeffizienz

- Energy-Delay Product

- $EDP = E * T_{exec} [Ws^2]$
- Nützliche Metrik zur Bestimmung der vollst. Effizienz eines Entwurfs, während eine bestimmte Funktion ausgeführt wird.
- Sinnvolles Maß bei Systemen, die mit maximaler Rate betrieben werden.

- bei Prozessoren:

- $[MIPS^2/\mu W]$
- $[SPEC^2/\mu W]$



Performanz

- Ausführungszeit T_{exec}
 - $T_{\text{exec}} = I_c * CPI * \tau = \frac{I_c * CPI}{f}$
 - I_c Instruction-Count eines Programms
 - CPI Cycles Per Instruction (Mittelwert)
 - τ Taktperiode = T_{clock}
 - f Taktfrequenz ($1/\tau$)
- Beispiel:
 - $I_c = 2000, CPI = 0.4, f = 400 \text{ MHz} \Rightarrow T_{\text{exec}} = 2 \mu\text{s}$

Performanz: Beispiel (I)

- MIPS-Rate (million instructions per second)

- $$MIPS = \frac{I_c}{T_{exec} \cdot 10^6} = \frac{f}{CPI \cdot 10^6}$$

- Beispiel:

- Prozessor mit einer Taktfrequenz von 500 MHz
- 3 Instruktionsklassen A, B, C mit $CPI_A = 1$, $CPI_B = 2$, $CPI_C = 3$
- 2 verschiedene Compiler erzeugen für das gleiche Programm folgende Instruktionszahlen I_c

	A	B	C
Compiler 1	$5 \cdot 10^9$	$1 \cdot 10^9$	$1 \cdot 10^9$
Compiler 2	$10 \cdot 10^9$	$1 \cdot 10^9$	$1 \cdot 10^9$
CPI	1	2	3

Software-Performanz: Beispiel (II)

- Taktzyklen: L

- Programm1: $(5 \cdot 1 + 1 \cdot 2 + 1 \cdot 3) \cdot 10^9 = 10 \cdot 10^9$
- Programm2: $(10 \cdot 1 + 1 \cdot 2 + 1 \cdot 3) \cdot 10^9 = 15 \cdot 10^9$
- Ausführungszeiten: L/f
- Programm 1: $(10 \cdot 10^9) / (500 \cdot 10^6) = 20 \text{ sec}$
- Programm 2: $(15 \cdot 10^9) / (500 \cdot 10^6) = 30 \text{ sec}$

- MIPS-Raten: $\sum \frac{I_c}{T_{exec} \cdot 10^6}$

- Programm1: $((5 + 1 + 1) \cdot 10^9) / (20 \cdot 10^6) = 350 \text{ MIPS}$
- Programm2: $((10 + 1 + 1) \cdot 10^9) / (30 \cdot 10^6) = 400 \text{ MIPS}$

- Programm 2 hat eine höhere MIPS-Rate, Programm 1 läuft schneller, da weniger Code.
- MIPS eher als Maß für die Komplexität eines Algorithmus, bzw. dessen Umsetzung.
- Nur T_{exec} ist wirklich aussagefähig.

	A	B	C
Programm 1	$5 \cdot 10^9$	$1 \cdot 10^9$	$1 \cdot 10^9$
Programm 2	$10 \cdot 10^9$	$1 \cdot 10^9$	$1 \cdot 10^9$
CPI	1	2	3



Performanz: Metriken

- MIPS (million instructions per second)
- MFLOPS (million floating-point operations per second)
- MACS (million multiply & accumulates per second)
 - wichtig bei DSPs
- MOPS (million operations per second)
 - alle Operationen zusammengezählt: ALUs, Adressrechnungen, DMA, ...
 - bei allen:
 - Paralleloperationen berücksichtigt
 - optimale Belegung vorausgesetzt
- Ausführungszeit
 - Compilation und möglichst viele Testläufe
 - ⇒ statistische Aussagen (Profiling und Tracing)

Profiling & Tracing

- Profiling und Tracing werden durchgeführt, indem an bestimmten Stellen zusätzlicher Monitoring-Code eingefügt wird
- Somit werden Protokolle über die Ausführung erstellt

Profiling & Tracing

- Profiling und Tracing werden durchgeführt, indem an bestimmten Stellen zusätzlicher Monitoring-Code eingefügt wird
- Somit werden Protokolle über die Ausführung erstellt
- Ein **Profil** enthält eine Menge von Ereignissen für die gesamte Ausführung eines Programms (ohne Timestamps)
 - Es ignoriert die zeitliche Abfolge der Ereignisse

Profiling & Tracing

- Profiling und Tracing werden durchgeführt, indem an bestimmten Stellen zusätzlicher Monitoring-Code eingefügt wird
- Somit werden Protokolle über die Ausführung erstellt
- Ein Profil enthält eine Menge von Ereignissen für die gesamte Ausführung eines Programms (ohne Timestamps)
 - Es ignoriert die zeitliche Abfolge der Ereignisse
- Ein **Trace** zeichnet die zeitliche Abfolge auf (mit Timestamps)
 - Die Datenmenge im Trace wächst mit der Laufzeit
 - Um den Speicherverbrauch beim Tracing zu begrenzen, müssen die Daten regelmäßig in den Speicher oder über das Netzwerk gesichert werden

Anwendungen

- **Profiling** wird verwendet für
 - Analyse der Rechenlast und kritischer Pfade
 - Aufteilung von Funktionen und Optimierungen im HW/SW Co-Design
- **Tracing** wird verwendet für
 - Detaillierte Analyse von Timing und Synchronisation in einem Programm
 - Daten- und Ereignisprotokollierung für die Simulation eines Rechensystems
 - Debugging

Optimal Profiling Instrumentation

- Wie instrumentieren wir den Code, um den Overhead zu minimieren?
 - „Optimally Profiling and Tracing Programs“ – Thomas Ball, James Larus
- QPT: Program profiler and tracing system
 - Instrumentierungsbasierte Profiling- und Tracing-Algorithmen
 - Profiling: Die Verzweigungs- und Ausführungshäufigkeiten jedes sequentiellen Programm-Basisblocks werden gemessen
 - Tracing: Die Ausführungsreihenfolge der Programm-Basisblöcke wird protokolliert
- Die so gewonnenen Daten ermöglichen die Berechnung der Ausführungskosten eines Programms

Begrenzungen der Hardware

- Wie schnell können wir rechnen?
 - Das Programm umfasst eine Menge an Operationen: *Computation FLOPs*
 - Unsere Hardware schafft begrenzte Menge pro Sekunde: *Accelerator FLOPs/s*
 - Wie lange dauert die Berechnung?

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}} \quad [10]$$

- **Achtung:**
 - Computation FLOPS misst **das Programm**
 - Accelerator FLOPS/s misst **die Hardware**

Begrenzungen der Hardware

- Wie schnell können wir rechnen?
- Wie schnell können wir Daten übertragen?
 - Für die Berechnung wird eine Menge von Daten gebraucht: *Communcation Bytes*
 - Wir haben begrenze Bandbreite pro Sekunde: *Memory Bandwidth Bytes/s*
 - Wie schnell ist die Übertragung?

$$T_{\text{comms}} = \frac{\text{Communcation Bytes}}{\text{Memory Bandwidth Bytes/s}}$$

[10]

Arithmetic Intensity

- Arithmetic Intensity zeigt ob Rechnungen oder Speicherzugriffe dominieren

$$\text{Arithmetic Intensity} = \frac{\text{Computation FLOPs}}{\text{Communcation Bytes}} \quad [10]$$

Menge an (floating point)
Operationen in dem Programm



Menge an Datenübertragung (Bytes)
für Ausführung des Programms



Roofline Model - Beispiel

- **Ziel:** Berechne $3 \cdot x + 4$

- **Ineffizient:**

- Lade x in ein Register → 1 Byte (laden)
- Multipliziere x mit 3 → 1 OP
- Speichere Ergebnis im Arbeitsspeicher → 1 Byte (schreiben)
- Lade das Ergebnis in ein neues Register → 1 Byte (laden)
- Addiere 4 zu dem Ergebnis → 1 OP
- Speichere das Endergebnis im Arbeitsspeicher → 1 Byte (schreiben)

2 OPs / 4 Bytes

Roofline Model - Beispiel

- **Ziel:** Berechne $3 \cdot x + 4$

- **Ineffizient:**

- Lade x in ein Register $\longrightarrow$ 1 Byte (laden)
- Multipliziere x mit 3 $\longrightarrow$ 1 OP
- Speichere Ergebnis im Arbeitsspeicher $\longrightarrow$ 1 Byte (schreiben)
- Lade das Ergebnis in ein neues Register $\longrightarrow$ 1 Byte (laden)
- Addiere 4 zu dem Ergebnis $\longrightarrow$ 1 OP
- Speichere das Endergebnis im Arbeitsspeicher $\longrightarrow$ 1 Byte (schreiben)

2 OPs / 4 Bytes

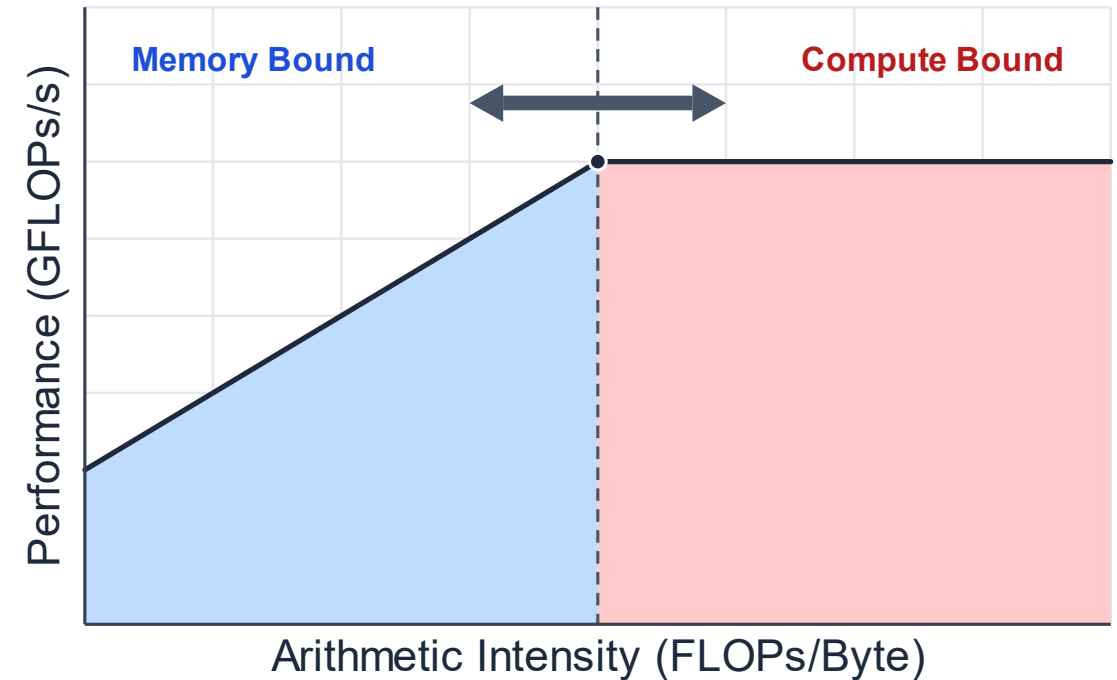
- **Besser:**

- Lade x in ein Register $\longrightarrow$ 1 Byte (laden)
- Multipliziere x mit 3 $\longrightarrow$ 1 OP
- Addiere 4 zu dem Ergebnis $\longrightarrow$ 1 OP
- Speichere das Endergebnis im Arbeitsspeicher $\longrightarrow$ 1 Byte (schreiben)

2 OPs / 2 Bytes

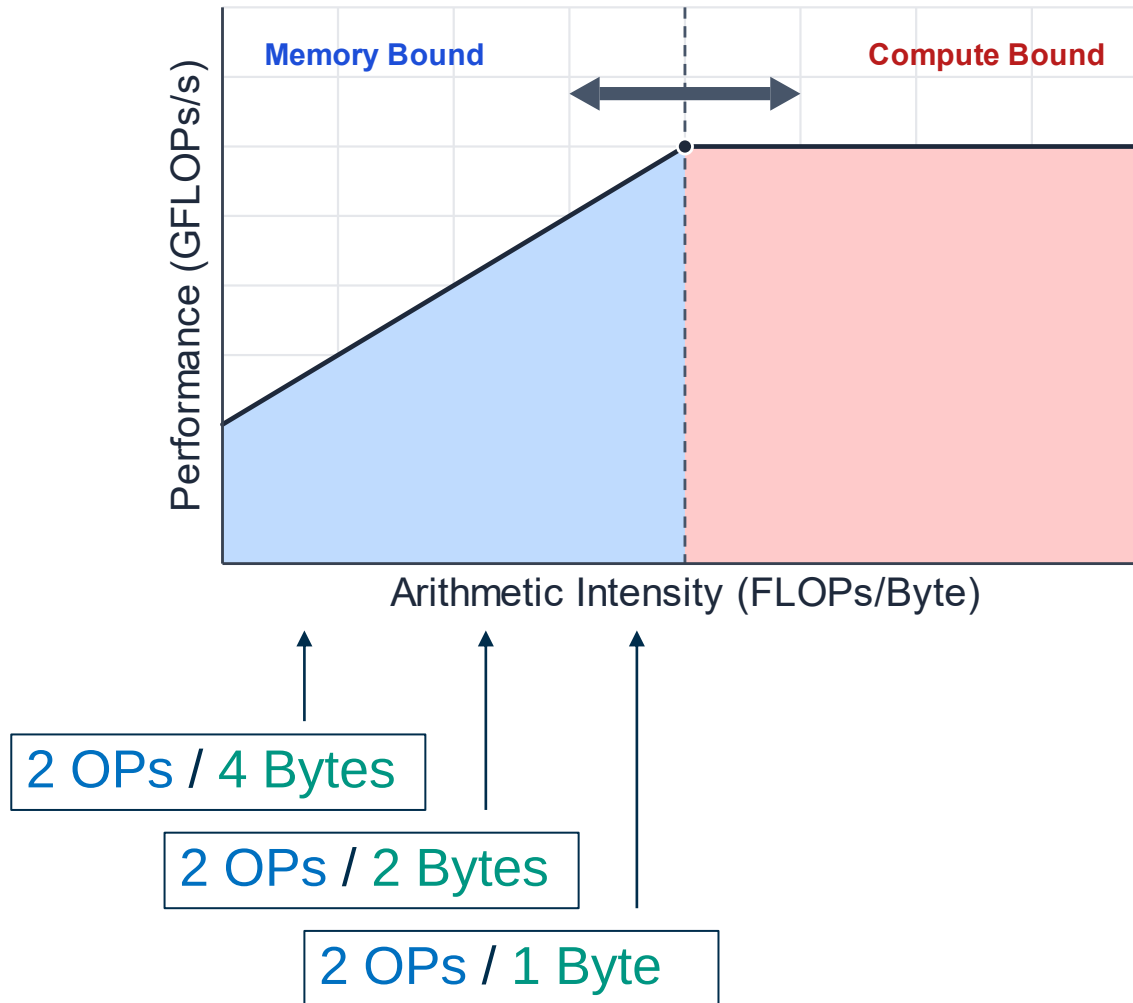
Roofline Model

- Zeigt ob die Programmausführung durch Speicherzugriffe oder Rechenleistung beschränkt ist
- Wenn mehr Operationen pro geladenem Byte ausgeführt werden können, wie verändert sich die Performance?



[10]

Roofline Model - Beispiel



- Höhere Intensity (Ops/Byte) aber keine höhere Performance?
 - Memory Bound
(Performance ist beschränkt durch Speicherzugriffe)
- Andernfalls
 - Compute Bound
(Performance ist beschränkt durch die Rechenleistung des Prozessors)

Quellen

- [1] The RISC-V Instruction Set Manual Volume I: Unprivileged ISA, Version 2025-05.08,
- [2] RISC-V processor specific application binary interface, Release 21.06.2025 <https://github.com/riscv-non-isa/riscv-elf-psabi-doc>
- [3] Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 1st edition,
- [4] Project F, RISC-V Assembler: Load Store <https://projectf.io/posts/riscv-load-store/> (Zugriff 21.04.2026)
- [5] RISC-V Webassembler SharpRISCV <https://rizwan3d.github.io/SharpRISCV/?ref=hackernoon.com> (Zugriff 21.04.2026)
- [6] RISC-V Assembly Programmer's Manual, Release 0.0.1 <https://github.com/riscv-non-isa/riscv-asm-manual/releases/tag/v0.0.1>
- [7] GCC Toolchain <https://gcc.gnu.org/> (Zugriff 21.04.2026)
- [8] Gnu Binutils <https://www.gnu.org/software/binutils/> (Zugriff 21.04.2026)
- [9] Computer systems: a programmer's perspective. Third Edition
- [10] JaxML, All About Rooflines <https://jax-ml.github.io/scaling-book/roofline/> (Zugriff 21.04.2026)
- [11] Panis, C.. (2004). Scalable DSP Core Architecture Addressing Compiler Requirements.

